



Republic of the Philippines
POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

COSC-E4
CS ELECTIVE 4 (Machine Learning)
Home Activity # 1

Name: Cyrus L. Jimenez Year and Section: 4-1

Assignment / Activity Rubric

Criteria Points	5pts	4pts	3pts	2pts
COMPLETION <i>How much of the task was completed?</i>	The entire task was completed with additional inputs.	100% of the task was completed	75% of the task was completed	At least 50% of the tasks was completed
ACCURACY <i>How much of the task was done right?</i>	The entire task (or almost all) of the task was done correctly.	75% of the task was done correctly.	At least 50% of the task was done correctly.	Less than 50% of the task was done correctly.
LEGIBILITY/ NEATNESS <i>Can the work pass the stranger test?</i>	Very neat. Writing illustrates a lot of thought and preparation.	Mostly neat and legible. Writing illustrates some thought and preparation.	Slightly legible. Writing illustrates little thought or preparation.	Not legible. Hard to read. Ideas expressed are difficult to understand.
DIRECTIONS <i>Did you follow directions? Was the task completed showing all work?</i>	The task was done showing lots of work neatly and easily followed.	The task was done showing most of the work.	The task was done showing some work.	The task was answers only.
EFFORT <i>The student worked at what level of their ability?</i>	Outstanding work, the student should be proud of their effort!	Good effort was made and the student met my expectations.	Some effort was made. The student just did enough work.	Minimal effort was given. The student should try harder.

Activity:

Perform research to demonstrate what Overfitting and Underfitting mean, including showcasing this through the use of **R**, **Python**, etc. Show an example using **R** or **Python** on how a Machine Learning Model becomes Overfit to the dataset, and how it becomes Underfit as well. Demonstrate it using R or Python, then explain each process and how the ML model became Overfit or Underfit to the dataset.

Prepared by.

Montaigne G. Molejon CS/IT
Faculty
CCIS-PUP Sta. Mesa

I. Introduction

Overview of Model Fitting in Machine Learning

Model fitting is the process of measuring how well a statistical model is able to describe a given set of observations (Institute of Data, 2024). In the field of data science, **models** serve as mathematical representations of real-world processes, and are built using various algorithms that are then applied or “fitted” to a collection of data points. This fitting process works by adjusting the model’s internal parameters in order to optimize the alignment between its predictions and the actual observed data (Institute of Data, 2024).

Model fitting plays a critical role in data analysis, as it enables data scientists to generate accurate predictions, uncover meaningful patterns, and arrive at well-informed decisions derived from the data. Besides from those, the quality of a model’s fit holds significant weight in determining how valid and reliable the conclusions drawn from that data ultimately are (Institute of Data, 2024).

Importance of Model Fitting

A well-fitted model does not necessarily align with every single data point, but instead **follows the general trend or curve** of the data, indicating that the model is neither underfit nor overfit. This balance is what makes a model useful and dependable in practice. When a model is not fitted properly, it becomes prone to producing inaccurate insights, rendering it unreliable and unsuitable for use in decision-making processes (*What Is Model Fitting and Why Is It Important?* | H2O.ai, n.d.). In other words, an ideal model finds the right spot by being sufficiently complex to recognize and capture genuine patterns within the data, without being so complex that it begins to memorize random noise (GeeksforGeeks, 2025).

II. Definition of Key Concepts

What is Underfitting?

Underfitting occurs when a model is **too simple to capture the true and important patterns** present in the data, resulting in poor performance on both training and testing data (GeeksforGeeks, 2025). Unlike overfitting where the model learns too much, an underfit model fails to learn enough, meaning it is unable to represent the underlying structure of the data in any meaningful way.

Several factors can lead to underfitting including having a model that is overly simplistic, applying very high regularization, working with weak or missing features, and not providing the model with sufficient training. A key concept tied to underfitting is **bias**, which refers to the error introduced when a model makes overly broad or incorrect assumptions about the data (GeeksforGeeks, 2025).

What is Overfitting?

Overfitting occurs when a model **learns too much from the training data**, capturing not only the true underlying patterns but also the noise and outliers present within it (GeeksforGeeks, 2025). As a result, while the model tends to perform exceptionally well on the training data, its

performance drops significantly when exposed to new, unseen test data, as it has essentially memorized the training set rather than truly learning from it.

This issue can arise due to a number of factors such as a model that is overly complex, contains too many features, or is trained on very little data is particularly susceptible to overfitting. Additionally, the absence of regularization techniques and having a high variance in the model can further contribute to this problem. A closely related concept is **variance**, which refers to the error that occurs when a model learns too much from the data, including its random noise, and high variance in a model is one of the key indicators that overfitting has taken place (GeeksforGeeks, 2025).

III. Demonstration of Underfitting and Overfitting with Regression

To better understand the concept of underfitting, overfitting, as well as ideal fitting, we will work with an open-ended project provided by CISCO in one of their courses about data science using Python. This project contains the linear regression model that will be reused together with the blood pressure dataset. All code implementation and execution will be performed in Google Colaboratory (Google Colab), so there will be no need to install packages locally and to reduce storage resources.

About the Dataset

Does blood pressure increase with age?

The 'blood-pressure-usa.csv' file contains systolic blood pressure readings from 100 individuals sampled by the National Health and Nutrition Examination Survey (NHANES) in the U.S., with each person measured three times.

Age	bp1	bp2	bp3
46	122	129	122
43	113	116	111
29	102	107	105
12	100	102	102
9	112	138	94
57	121	121	123
11	106	103	103
23	110	104	106
34	143	138	153
32	100	101	100

Figure 1. First 10 rows of blood pressure dataset.

Linear Regression Model

The linear regression model provided in the CISCO project serves as the foundation for this demonstration. To better illustrate the concepts of underfitting, ideal fitting, and overfitting, the model will be extended to polynomial regression later on. By increasing the complexity of the

model, it becomes possible to observe how model performance changes and to compare the fitting behavior of simple and complex regression models.

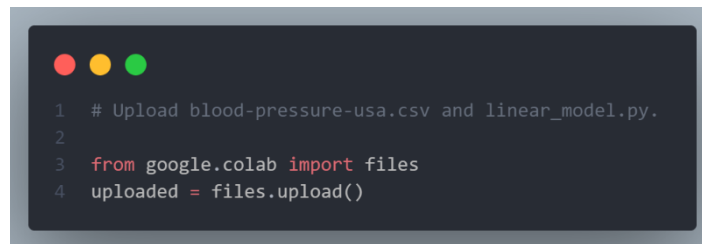
```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import r2_score
5
6 class LinearModel:
7     def __init__(self, model_name=""):
8         self.model_name = model_name
9
10    def fit(self, x, y):
11        x = pd.DataFrame(x)
12        linear_model = LinearRegression().fit(x, y)
13        y_pred = linear_model.predict(x)
14        self.slope = linear_model.coef_[0]
15        self.intercept = linear_model.intercept_
16        self.rsquared = r2_score(y, y_pred)
17        return self
18
19    def predict(self, x):
20        return self.slope * x + self.intercept
21
22    def plot_model(self, x_min, x_max, color="black"):
23        y_min = self.predict(x_min)
24        y_max = self.predict(x_max)
25        plt.plot([x_min, x_max], [y_min, y_max], color=color)
26
27    def print_model_info(self):
28        m = self.slope
29        b = self.intercept
30        rsquared = self.rsquared
31        model_name = self.model_name
32        print(f'LinearModel({model_name}):')
33        print(f'Slope: {m}')
34        print(f'Intercept: {b}')
35        print(f'R-squared: {rsquared}')
```

Figure 2. Code snippet of the linear regression model from CISCO.

The succeeding figures present the implementation of the regression model in Google Colab. Each code cell illustrates a stage of the regression workflow, including data preparation, model training, visualization, and evaluation. The resulting models are then compared to illustrate how model complexity influences fitting behavior and prediction performance across underfitting, ideal fitting, and overfitting scenarios.

Uploading the Necessary Files

The first step uploads the blood pressure dataset and the linear regression model file to the Google Colab environment, making them available for subsequent analysis and model implementation. Executing the code cell prompts the user to upload the required files.

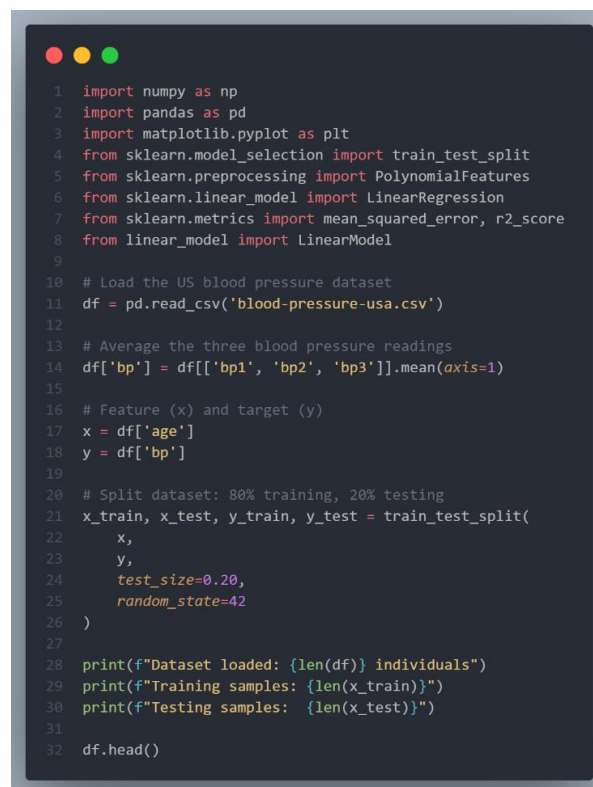
A screenshot of a Google Colab code cell with a dark background and light-colored text. The code is as follows:

```
1 # Upload blood-pressure-usa.csv and linear_model.py.
2
3 from google.colab import files
4 uploaded = files.upload()
```

Figure 3. Code cell for uploading dataset and model files.

Importing Libraries and Loading the Dataset

This cell imports the required Python libraries for data manipulation, visualization, and regression modeling. It loads the blood pressure dataset, extracts the age and systolic blood pressure variables, and partitions the data into training and testing sets to support model training and performance evaluation. The output prints the partitioning of the data as well as the first 5 rows of the data frame with the new column for the computed mean blood pressure.

A screenshot of a Google Colab code cell with a dark background and light-colored text. The code is as follows:

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import PolynomialFeatures
6 from sklearn.linear_model import LinearRegression
7 from sklearn.metrics import mean_squared_error, r2_score
8 from linear_model import LinearModel
9
10 # Load the US blood pressure dataset
11 df = pd.read_csv('blood-pressure-usa.csv')
12
13 # Average the three blood pressure readings
14 df['bp'] = df[['bp1', 'bp2', 'bp3']].mean(axis=1)
15
16 # Feature (x) and target (y)
17 x = df['age']
18 y = df['bp']
19
20 # Split dataset: 80% training, 20% testing
21 x_train, x_test, y_train, y_test = train_test_split(
22     x,
23     y,
24     test_size=0.20,
25     random_state=42
26 )
27
28 print(f"Dataset loaded: {len(df)} individuals")
29 print(f"Training samples: {len(x_train)}")
30 print(f"Testing samples: {len(x_test)}")
31
32 df.head()
```

Figure 4. Code cell for importing libraries, dataset loading, and train-test splitting.

```
Dataset loaded: 100 individuals
Training samples: 80
Testing samples: 20
```

	age	bp1	bp2	bp3	bp
0	46	122	129	122	124.333333
1	43	113	116	111	113.333333
2	29	102	107	105	104.666667
3	12	100	102	102	101.333333
4	9	112	138	94	114.666667

Figure 5. Output showing train-test split results and dataset preview.

Visualizing the Entire Dataset

This cell generates a scatter plot of the complete dataset to provide an initial view of the relationship between age and systolic blood pressure before any model training is performed.

```
1 plt.figure(figsize=(10, 6))
2
3 plt.scatter(x, y, color='steelblue', alpha=0.6, s=30)
4
5 plt.xlabel('Age')
6 plt.ylabel('Blood Pressure')
7 plt.title('Blood Pressure vs Age')
8
9 plt.grid(True, alpha=0.3)
10 plt.show()
```

Figure 6. Code cell for dataset visualization with scatter plot.

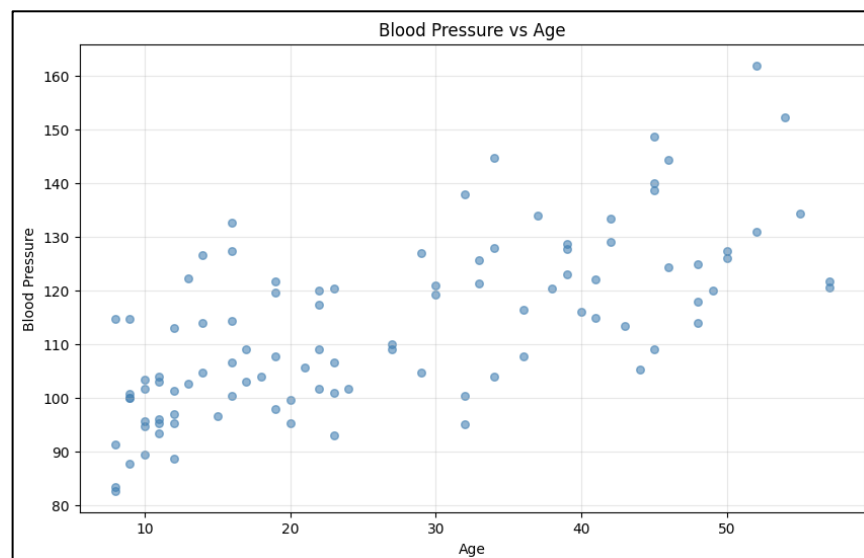


Figure 7. Scatter plot of age versus mean blood pressure.

Building the Polynomial Regression Model

This cell extends the original linear regression model provided by CISCO to support polynomial regression of varying degrees, enabling comparisons between models with different levels of complexity.

```
1 class PolynomialModel(LinearModel):
2     """
3     Extends LinearModel to support polynomial regression of any degree.
4     Inherits model_name, fit(), rsquared, and print_model_info() from LinearModel.
5     """
6
7     def fit_poly(self, x, y, degree):
8         """Transform x into polynomial features, then fit using LinearModel."""
9         self.degree = degree
10        self.poly = PolynomialFeatures(degree=degree, include_bias=False)
11        x_poly = self.poly.fit_transform(x.values.reshape(-1, 1))
12
13        # Fit via parent LinearModel (stores slope of first feature + intercept + R^2)
14        super().fit(x_poly, y)
15
16        # Also store the full coefficient array for multi-feature prediction
17        lr = LinearRegression().fit(x_poly, y)
18        self.full_coef = lr.coef_
19        return self
20
21    def predict_range(self, x_vals):
22        """Predict bp for a range of age values using all polynomial coefficients."""
23        x_poly = self.poly.transform(x_vals.reshape(-1, 1))
24        return x_poly @ self.full_coef + self.intercept
25
26    # Fit the three models
27    model_underfit = PolynomialModel(model_name="Underfit - Degree 1").fit_poly(x_train, y_train, degree=1)
28    model_goodfit = PolynomialModel(model_name="Good Fit - Degree 4").fit_poly(x_train, y_train, degree=4)
29    model_overfit = PolynomialModel(model_name="Overfit - Degree 20").fit_poly(x_train, y_train, degree=20)
30
31    print("All three models fitted successfully.")
```

Figure 8. Code cell for defining a polynomial model builder.

Plotting Individual Regression Models

This cell visualizes each regression model separately to allow the fitting behavior of different polynomial degrees to be examined independently.

```
1 # Generate a smooth range of age values
2 x_range = np.linspace(
3     min(x_train.min(), x_test.min()),
4     max(x_train.max(), x_test.max()),
5     500
6 )
7
8 plot_configs = [
9     (model_underfit, '#e74c3c', '--', 'Underfitting - Degree 1\n(Too Simple: straight line misses the curve)'),
10    (model_goodfit, '#2ecc71', '-', 'Good Fit - Degree 4\n(Balanced: follows the real trend)'),
11    (model_overfit, '#e67e22', '--', 'Overfitting - Degree 20\n(Too Complex: wiggles to chase every point)'),
12 ]
13
14 # Create a figure with three side-by-side subplots
15 fig, axes = plt.subplots(1, 3, figsize=(18, 9))
16
17 # Overall title
18 fig.suptitle(
19     'Predicting Blood Pressure from Age - Underfit vs Good Fit vs Overfit',
20     fontsize=13, fontweight='bold', y=1.02
21 )
22
23 for ax, (model, color, linestyle, title) in zip(axes, plot_configs):
24     y_curve = model.predict_range(x_range)
25
26     ax.scatter(x_train, y_train, color='steelblue', alpha=0.6, s=30, label='Training Data')
27     ax.scatter(x_test, y_test, color='red', alpha=0.6, s=30, markers='x', label='Test Data')
28     ax.plot(
29         x_range, y_curve,
30         color=color, linewidth=2.5, linestyle=linestyle,
31         label=f'Model Curve (R^2={model.rsquared:.4f})'
32     )
33     ax.set_title(title, fontsize=10, fontweight='bold', pad=10)
34     ax.set_xlabel('Age', fontsize=10)
35     ax.set_ylabel('Systolic Blood Pressure (mmHg)', fontsize=10)
36     ax.legend(fontsize=9)
37     ax.grid(True, linestyle='--', alpha=0.4)
38
39 plt.tight_layout()
40 plt.savefig('separate_plots.png', dpi=150, bbox_inches='tight')
41 plt.show()
42 print("Saved: separate_plots.png")
```

Figure 9. Code cell for creating separate plots of the regression models.

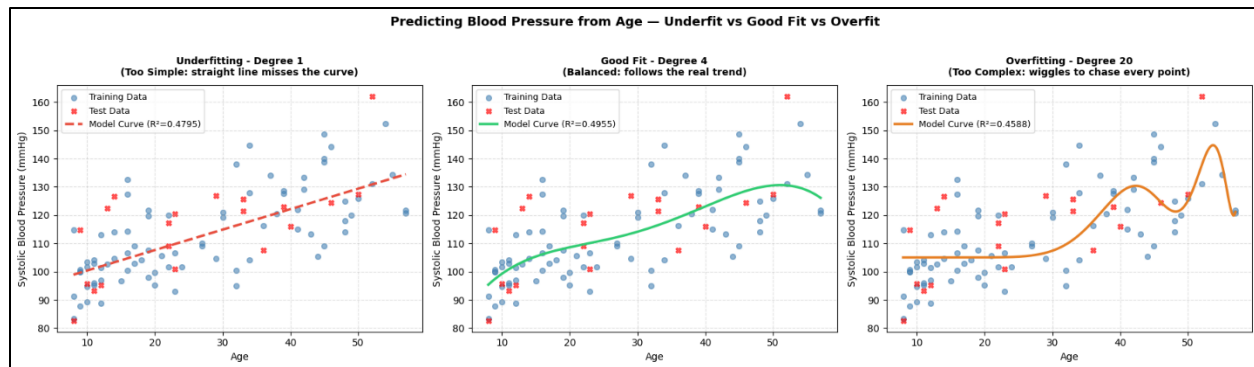


Figure 10. Separate plots of the underfitting, good fit, and overfitting regression models.

The first model uses a polynomial degree of 1, making it the least complex among the three models. As shown in the plot, the regression line captures the overall upward trend between age and systolic blood pressure, which is expected given that the data points show a slight positive relationship. However, the model fails to capture the subtle curvature present in the data and instead represents the relationship as a straight line. By oversimplifying the underlying pattern, the model exhibits high bias and limited flexibility, causing it to underfit the dataset. As a result, important information contained in the data is not fully represented by the model.

The second model uses a polynomial degree of 4, providing a moderate level of complexity. Unlike the degree-1 model, it is able to follow both the overall upward trend and the slight curvature observed in the data. The resulting regression curve closely reflects the underlying pattern without introducing excessive fluctuations. This balance between bias and variance allows the model to generalize well to unseen data while still capturing meaningful relationships within the training set. Among the three models, it achieves the highest R^2 value, indicating the strongest goodness of fit for this particular dataset and making it the most appropriate model for prediction.

The third model uses a polynomial degree of 20, making it the most complex model in the comparison. The regression curve exhibits a highly wavy pattern, particularly for ages above 40, suggesting that the model attempts to follow individual training data points too closely. Rather than learning the true underlying relationship between age and blood pressure, the model has effectively memorized the training data, including its random fluctuations. This behavior is characteristic of overfitting and is associated with high variance. Although highly complex models can fit training data very closely, they often struggle to generalize to unseen observations because they have learned patterns that are specific to the training set rather than the true underlying trend.

Comparing Regression Models on a Single Plot

This cell overlays the regression curves on a common graph to facilitate direct comparison of model complexity and fitting behavior. Plotting the models against one another on the same set of axes provides a clearer visualization of their differences, which allows the effects of increasing and decreasing model complexity on the fitted curves to be more easily observed.

```

1 plt.figure(figsize=(11, 6))
2
3 # Scatter: actual data points
4 plt.scatter(x_train, y_train, color='steelblue', alpha=0.45, s=30, Label='Training Data', zorder=2)
5 plt.scatter(x_test, y_test, color='red', alpha=0.45, s=30, marker='X', Label='Test Data', zorder=2)
6
7 # Each model curve
8 combined_configs = [
9     (model_underfit, '#e74c3c', '--', f'Underfit - Degree 1 (R²={model_underfit.rsquared:.4f})'),
10    (model_goodfit, '#2ecc71', '-', f'Good Fit - Degree 4 (R²={model_goodfit.rsquared:.4f})'),
11    (model_overfit, '#e67e22', '-', f'Overfit - Degree 20 (R²={model_overfit.rsquared:.4f})'),
12 ]
13
14 for model, color, linestyle, label in combined_configs:
15     y_curve = model.predict_range(x_range)
16     plt.plot(x_range, y_curve, color=color, linewidth=2.5, linestyle=linestyle, Label=label, zorder=3)
17
18 plt.title('Blood Pressure vs Age - Model Comparison', fontsize=13, fontweight='bold')
19 plt.xlabel('Age', fontsize=11)
20 plt.ylabel('Systolic Blood Pressure (mmHg)', fontsize=11)
21 plt.legend(fontsize=10)
22 plt.grid(True, linestyle='--', alpha=0.4)
23 plt.tight_layout()
24 plt.savefig('combined_plot.png', dpi=150, bbox_inches='tight')
25 plt.show()
26 print("Saved: combined_plot.png")

```

Figure 11. Code cell for creating a combined comparison plot.

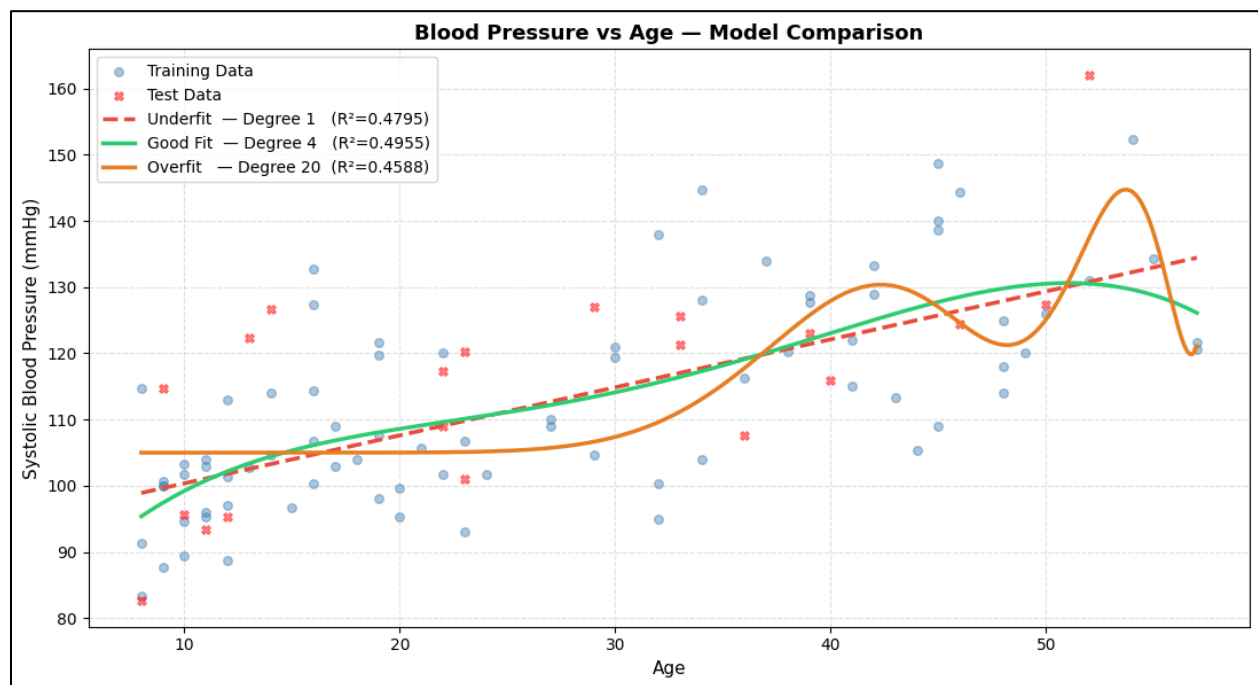


Figure 12. Comparison of underfitting, good fit, and overfitting regression models on a single plot.

Summarizing Model Performance Metrics

This cell presents a summary table containing the R-squared (R^2) and Mean Squared Error (MSE) values for each model on both the training and testing datasets.

```

1 print("=" * 85)
2 print(f"{'Model':<25} {'Train R²':>10} {'Test R²':>10} {'Train MSE':>15} {'Test MSE':>15}")
3 print("=" * 85)
4
5 for model in [model_underfit, model_goodfit, model_overfit]:
6
7     y_pred_train = model.predict_range(x_train.values)
8     y_pred_test = model.predict_range(x_test.values)
9
10    train_r2 = r2_score(y_train, y_pred_train)
11    test_r2 = r2_score(y_test, y_pred_test)
12
13    train_mse = mean_squared_error(y_train, y_pred_train)
14    test_mse = mean_squared_error(y_test, y_pred_test)
15
16    print(f"{'model.model_name':<25} {'train_r2':>10.4f} {'test_r2':>10.4f} {'train_mse':>15.4f} {'test_mse':>15.4f}")
17
18 print("=" * 85)
19 print()
20
21 for model in [model_underfit, model_goodfit, model_overfit]:
22     model.print_model_info()
23     print(f'Polynomial Degree: {model.degree}')
24     print('-' * 60)

```

Figure 13. Code cell for displaying model performance and model information.

```

=====
Model                                Train R²    Test R²    Train MSE    Test MSE
=====
Underfit – Degree 1                   0.4795     0.4232     125.9756     161.1558
Good Fit – Degree 4                   0.4955     0.4327     122.0806     158.5023
Overfit – Degree 20                   0.4588     0.3414     130.9805     184.0303
=====

LinearModel(Underfit – Degree 1):
Slope: 0.7250818184993348
Intercept: 93.1117512793138
R-squared: 0.47945443104923036
Polynomial Degree: 1
-----

LinearModel(Good Fit – Degree 4):
Slope: 5.418766981846327
Intercept: 66.36971662377181
R-squared: 0.4955489655610409
Polynomial Degree: 4
-----

LinearModel(Overfit – Degree 20):
Slope: 2.695357237741357e-44
Intercept: 104.99524361553972
R-squared: 0.4587736308352933
Polynomial Degree: 20
-----

```

Figure 14. Output showing model performance and model information.

As shown in the table, the good fit model (Degree 4) achieved the highest R^2 values on both the training set (0.4955) and test set (0.4327), indicating the strongest goodness of fit among the three models. The underfitting model (Degree 1) produced slightly lower R^2 values, suggesting that while it was able to capture the general relationship between age and blood pressure, its limited complexity prevented it from fully representing the underlying pattern in the data. In contrast, the overfitting model (Degree 20) obtained the lowest R^2 values on both datasets, indicating weaker performance in terms of goodness of fit. A similar trend can be observed in the Mean Squared Error (MSE) results, where the good fit model achieved the lowest training and test MSE values (122.0806 and 158.5023, respectively), signifying more accurate predictions. The underfitting model recorded slightly higher MSE values, whereas the overfitting model produced

the highest errors, especially on the test set (184.0303), indicating that it performed poorly on unseen data. Overall, these results demonstrate that the degree-4 model provides the best balance between model complexity and predictive performance, while the degree-1 model suffers from underfitting and the degree-20 model exhibits overfitting behavior that negatively affects its ability to generalize to new data.

IV. Conclusion

Going back to the main question raised at the start of this activity: does blood pressure increase with age? Based on the scatter plot of the dataset, it can be observed that there is a general upward trend, meaning that as a person gets older, their blood pressure also tends to rise. However, the increase is gradual and not dramatic. It is also noticeable that the data points are not steeply diagonal, and this is because there are still people who, even at an older age, manage to maintain a normal blood pressure level. This suggests that age alone is not the only factor that affects blood pressure, as other elements such as lifestyle, diet, and genetics also play a role in determining a person's blood pressure.

As for why underfitting and overfitting occur in the models demonstrated in this activity, it all comes down to model complexity and the imbalance between bias and variance. In the case of this specific dataset, the underfit model used a degree-1 polynomial, which was too simple to capture the gradual curve in the relationship between age and blood pressure, resulting in high bias. On the other hand, the overfit model used a degree-20 polynomial, which was far too complex for the size and nature of the dataset, causing the model to chase noise and random fluctuations in the training data instead of learning the real pattern, resulting in high variance. The good fit model with degree-4 struck the right balance, being complex enough to follow the general trend without memorizing the noise.

V. References

Dcl-Docs. (n.d.). *dcldata/data-raw/blood_pressure at master · dcl-docs/dcldata*. GitHub. https://github.com/dcl-docs/dcldata/tree/master/data-raw/blood_pressure

GeeksforGeeks. (2025, December 10). *Underfitting and overfitting in ML*. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/underfitting-and-overfitting-in-machine-learning/>

Institute of Data. (2024, January 11). *Understanding model fitting in Data Science* | Institute of Data. *Institute of Data*. <https://www.institutedata.com/blog/model-fitting-in-data-science/>

What Is Model Fitting and Why Is It Important? | H2O.ai. (n.d.). <https://h2o.ai/wiki/model-fitting/>